

P1

Introduction

For this assignment, the task is to learn how an OS is initially setup. The given pre-code represents a copy of a barebone OS on a physical disc. The actual compiled file is in ISO format. Three components are presents in the compiled ISO file, the GRUB bootloader, the kernel and CPIO filesystem. The project is consisted of two parts; the first one is about locating the CPIO filesystem within the boot image and the second part is about loading the processes into the memory and run them on command.

Technical background

Firmware and BIOS [1]

A firmware is a pre-written program embedded to the motherboard. BIOS is a firmware that runs at the start when a computer first activate. Its first job is to search for the other data components, such as RAM, and initializes them. It does so by searching the PCIe (i.e serial bus system for transferring data) for any detectable devices. After that, it finds the boot device and loads the boot loader into the memory. The boot loader then loads the kernel into the memory from the boot device and then it gets written back. The kernel, the core of the OS, then does its tasks like initializing drivers and establishing connection with the hardware.

Bootloader and GRUB [1]

Bootloader is a program what get loaded into the main memory after the BIOS has checked for the other data components. The job of the bootloader is to locate the kernel within the boot device and load it into the main memory. It knows where location of every. After the job is done, the bootloader is written back to the storage unit that it originally came from. An example of a bootloader is GRUB, which is used in this assignment.

Kernel

The kernel is the part of the OS that performs all of the tasks of an OS, such as system calls, resource management and so on.

Operating system [2]

Operating system is a software that offers services that allows for a smooth user experience. One of those services are resource management. It manages processes behind the curtain so that it does not cause any issues for the hardware and the running applications. One example is writing and reading of data from memory to storage device and vice versa. The OS has to know when and how to perform such actions, and where those data are stored. The other service is to wrap the functionalities of the hardware with abstractions to make it less difficult for users to use the computer. For example, a user can simply save a file to a hard disk with a click of a button on the screen. The process is a lot more difficult behind the scenes. The file has to be written from the memory (assuming that it was opened) to the hard disk. The content of the file has to travel via the bus system to the hard disk. The hard disk has to move its arm to write the content and so on. The core of every OS is called a kernel; every OS has a filesystem and display UI for ease of use. Those are the three parts of an OS.

Memory hierarchy and the main memory

The main memory, or RAM to be more specific, is a data component that stores segments of different programs during runtime. To run a program, it has to be divided into segments and only the relevant segments will be loaded into the main memory. There are many types of storage units in the world of computers. One could organize those storage units into a hierarchy, and the higher up a storage unit is placed, the faster the read, write and seek operations will be. While the further down the hierarchy one moves, those storage units will be slower, but larger. It is a tradeoff between space and speed. Main memory

Implementation and design

Loading the filesystem

As mentioned in technical background, GRUB's job is to locate and write the kernel into the main memory. It also has the information of where the CPIO filesystem is located. The solution is to include that information in the "boot_info" variable so that it can be loaded into the main memory.

The implementation consisted of a code paragraph in read_boot_info function that gives the information to the "boot_info" variable when the right tag type occurs.

Tests and results

Conclusion

The implementation only covers the first objective of the project. The solution is considered unfinished.

Sources

- 1) Bos, Herbert &. Tanenbaum, Andrew. Modern Operating systems fifth edition. chapter 1.3.6. Pearson education inc.
- 2) Bos, Herbert &. Tanenbaum, Andrew. Modern Operating systems fifth edition. chapter 1.1 Pearson education inc.